Multivariate Time-Series Prediction Using Deep Learning

Appliance Energy Prediction

ABINAYA R

3/27/26

Deep Learning
Time-Series

Table of Contents

1. Introduction
2. Data Insights
3. Preprocessing
4. Feature Engineering
5. Model Design
6. Results
7. Model Optimization
8. Challenges and Solutions
9. Conclusion
10. References

1. Introduction

Household energy consumption is one of those problems that looks simple on the surface but gets complicated quickly once you start working with the data. Appliances switch on and off unpredictably, consumption spikes during cooking and drops during the night, and the same hour on a Monday looks completely different from the same hour on a Saturday. Building a model that handles all of that reliably is genuinely challenging, and that is what this project set out to do.

The dataset used here is the Appliance Energy Prediction Dataset, collected from a real low-energy house in Belgium over roughly four and a half months in 2016. It contains nearly 20,000 sensor readings taken every 10 minutes, covering indoor temperature and humidity across multiple rooms, outdoor weather conditions, lighting energy, and several time-based indicators. The goal is to predict the Appliances column — total household appliance energy consumption in watt-hours (Wh).

My approach was to build a full machine learning pipeline from scratch: starting with careful exploratory analysis, cleaning the data, engineering meaningful features, and then training and comparing five different models. Two classical baselines were included — Linear Regression and Random Forest — alongside three deep learning architectures: a stacked LSTM with Batch Normalization, a GRU, and a CNN-LSTM hybrid. Every design decision was made deliberately, and this report explains the reasoning behind each one honestly, including where the results were surprising.

2. Data Insights

2.1 Dataset Overview

Property	Value
Total records	19,735
Sampling interval	10 minutes
Date range	January 11, 2016 — May 27, 2016
Target variable	Appliances (Wh)
Original columns	29
Missing values	0
Noise columns removed	rv1, rv2
Working columns after cleaning	27

The dataset was in excellent condition from the start — not a single missing value across nearly 20,000 rows. That is rare for real sensor data and meant preprocessing effort could be focused on outliers, noise removal, and feature construction rather than imputation strategies.

2.2 Target Variable Analysis

Statistic	Value
Mean	97.69 Wh
Standard deviation	102.52 Wh
Minimum	10 Wh
25th percentile	50 Wh
Median	60 Wh
75th percentile	100 Wh
Maximum	1,080 Wh

The target variable is heavily right-skewed. The median sits at 60 Wh while the mean is pulled up to nearly 98 Wh by high-consumption spikes. This gap between median and mean tells a familiar household story: appliances are idle or lightly used most of the time, but occasionally the oven, washing machine, and heating system all run together and consumption spikes sharply. Standard deviation of 102.52 Wh relative to a mean of 97.69 Wh confirms just how much variability there is — this is not a smooth, predictable signal.

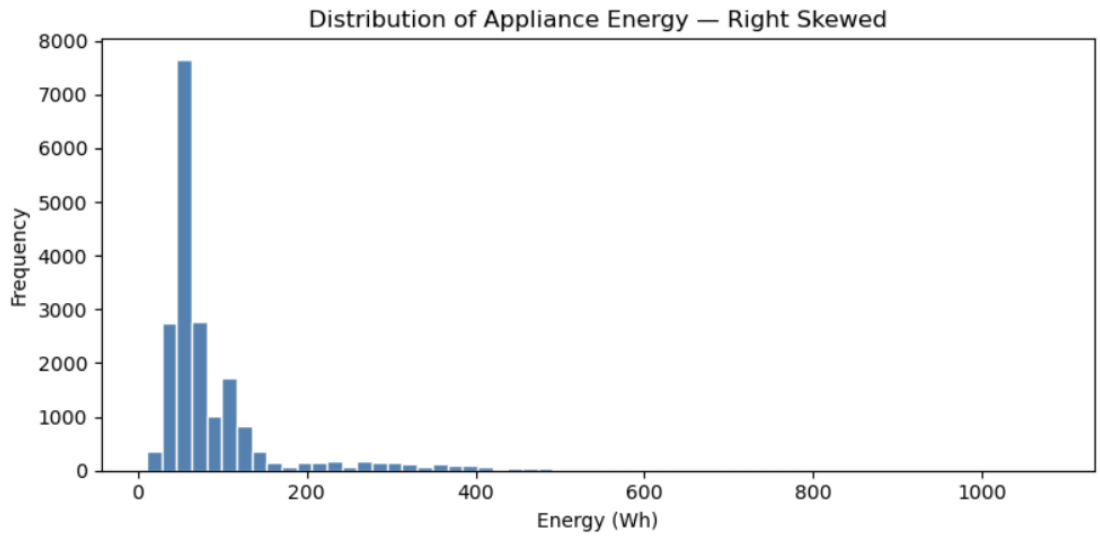


Figure 1: Distribution of Appliance Energy Consumption — Right Skewed

2.3 EDA Findings

Energy over time: Plotting consumption across the full four-month window reveals a noisy but structured signal. There is no single dominant trend — instead the data oscillates daily with clear peaks and troughs. A visual inspection also shows that winter months have noticeably different consumption patterns compared to the April and May readings, reflecting seasonal changes in heating load.

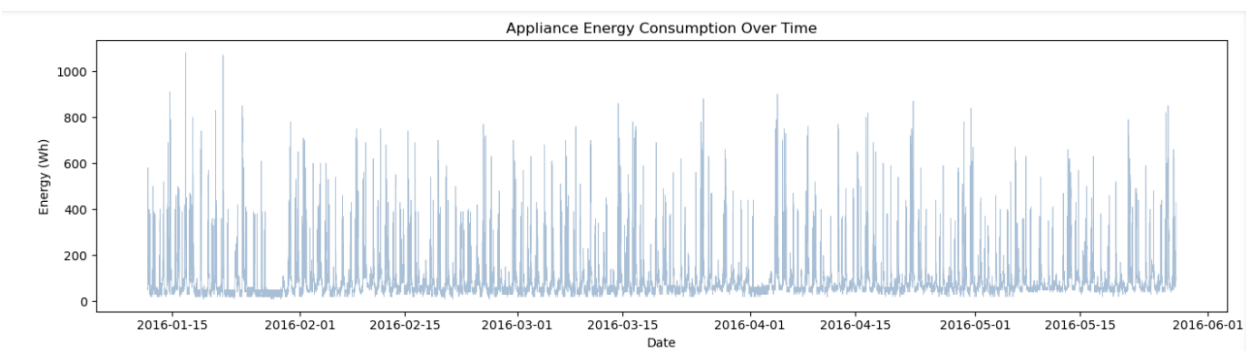


Figure 2: Appliance Energy Consumption Over Time (January to May 2016)

Hourly patterns: Averaging consumption by hour of day reveals a consistent daily structure. A morning peak appears between 07:00 and 09:00 driven by breakfast routines and morning activity. Consumption dips during midday working hours, then rises again in the evening between 18:00 and 21:00 — the household's busiest period for cooking, laundry, and entertainment. Overnight readings drop to their lowest levels.

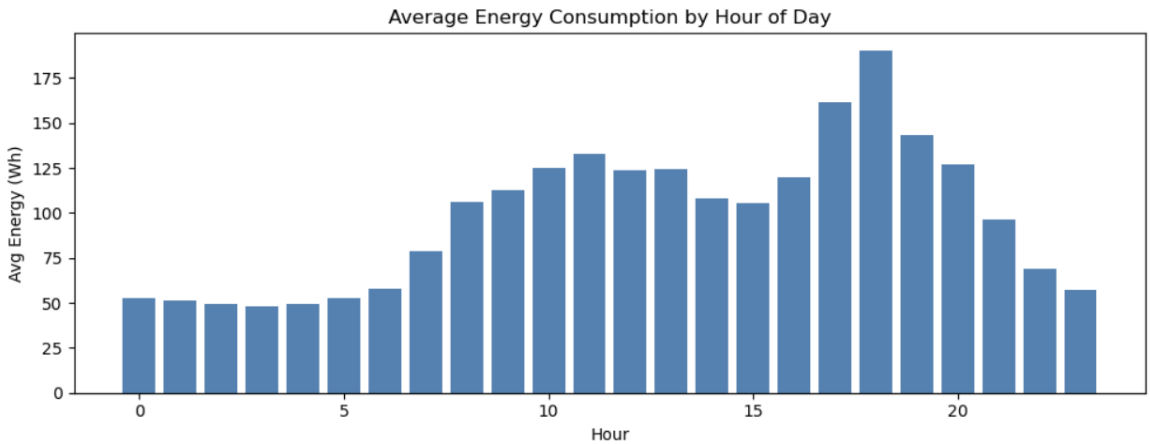


Figure 3: Average Energy Consumption by Hour of Day

Day of week patterns: Boxplots by day of week show that weekends carry higher median consumption than weekdays. The explanation is straightforward — when occupants are home during the day rather than at work, more appliances run throughout the full day rather than just morning and evening.

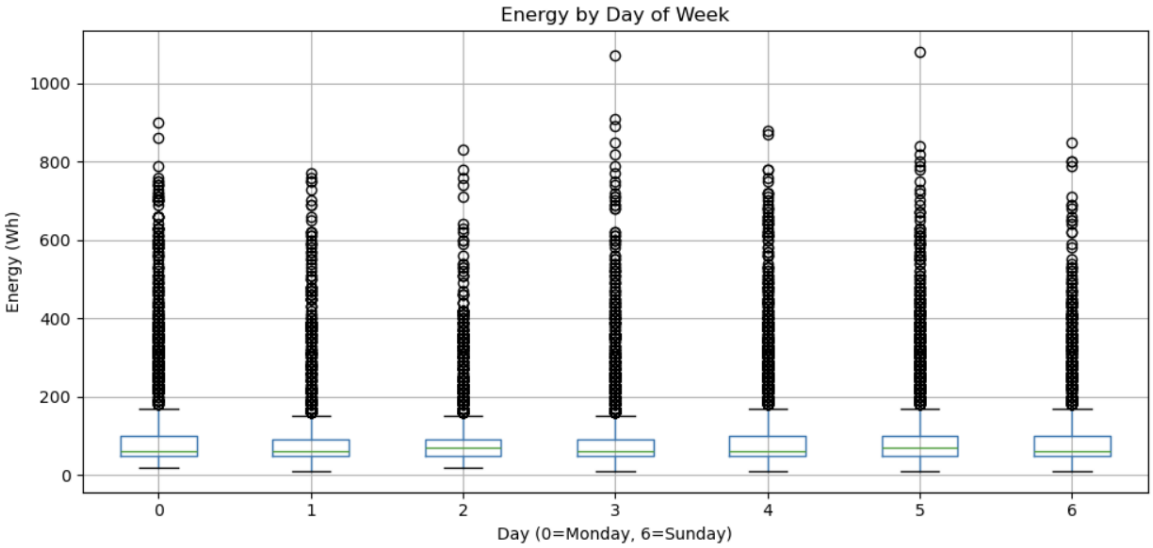


Figure 4: Appliance Energy Consumption by Day of Week

Correlation structure: Indoor temperature sensors T1 through T6 show moderate positive correlations with appliance consumption. The lights feature shows a notably strong correlation, which makes sense since lighting and appliance use tend to occur together during active household

periods. Outdoor conditions like T_out and RH_out have weaker but nonzero correlations, reflecting the indirect effect of weather on heating and cooling loads.

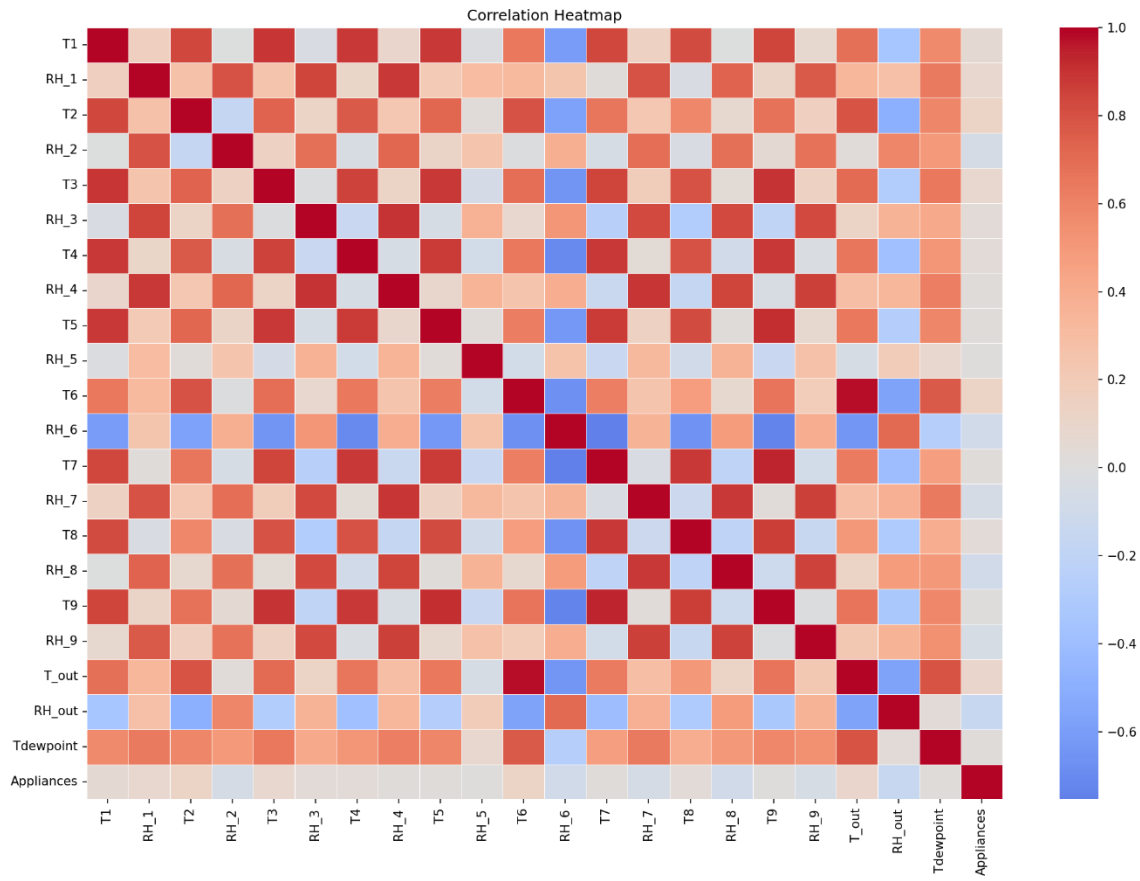


Figure 5: Correlation Heatmap — Temperature, Humidity and Target Variable

ACF and PACF analysis: The autocorrelation function showed strong autocorrelation at short lags (1 to 6 steps) with gradual decay, confirming that recent consumption history is the most powerful predictor of the next value. The PACF showed that most of the direct predictive power is concentrated in the first one to three lags. This analysis directly informed both the lag feature selection and the choice of a 36-step lookback window for sequence models.

2.4 The Noise Variable Discovery

Columns rv1 and rv2 appear in the dataset as though they are real features. On closer inspection, they turn out to be identical across every single one of the 19,735 rows — confirmed programmatically with rv1 == rv2 returning True universally. Their correlation with the target variable is essentially zero at −0.011. These are intentional random noise injections included in the dataset as a quality check. Identifying and removing them was important: keeping them would add pure noise to model inputs and could mislead feature importance analysis in tree-based models.

2.5 Feature Importance from Random Forest

After feature engineering, a Random Forest was used to rank the predictive value of all 47 engineered features. The top 10 were:

Rank	Feature	Importance
1	lag_1	0.6174
2	rolling_mean_6	0.0212
3	rolling_std_6	0.0152
4	rolling_std_18	0.0136
5	rolling_mean_18	0.0134
6	lag_3	0.0134
7	lag_12	0.0131
8	NSM	0.0128
9	Press_mm_hg	0.0124
10	rolling_std_36	0.0118

The most striking finding here is that lag_1 alone accounts for 61.7% of the total feature importance — far more than every other feature combined. This means that knowing what appliance energy consumption was 10 minutes ago is by far the best predictor of what it will be in the next 10 minutes. This result has significant implications for model design and helps explain the final performance rankings, which are discussed in the Results section.

3. Preprocessing

3.1 Missing Values

A complete missing value check was performed across all 29 columns. The result was zero missing values throughout the entire dataset. No imputation was needed and none was applied. This was confirmed explicitly before proceeding to any other step.

3.2 Noise Column Removal

Columns rv1 and rv2 were dropped after confirming they were identical across all rows and had near-zero correlation with the target. The working dataset after removal contained 27 columns.

3.3 Outlier Detection and Treatment

IQR analysis was applied to the target variable:

Parameter	Value
Q1	50 Wh
Q3	100 Wh
IQR	50 Wh
Upper bound (1.5×IQR)	175 Wh
Outliers identified	2,138 records (10.83%)
99th percentile	576.6 Wh

Over 10% of records fall above the IQR upper bound of 175 Wh. The critical question is whether these should be removed or kept. In this dataset they represent genuine physical events — high-energy appliance usage like cooking, running the dishwasher, or heating the house on a cold morning. Removing them would cause the model to never learn these patterns, leading to systematic underprediction during the moments that matter most in real-world use.

The chosen approach was 99th percentile capping at 576.6 Wh. Values above this threshold — less than 1% of all records, representing the genuinely extreme tail — were capped to reduce their disproportionate influence on the training loss. All realistic high-consumption events were preserved. A separate capped column (Appliances_capped) was created and used as the training target, preserving the original Appliances column for reference.

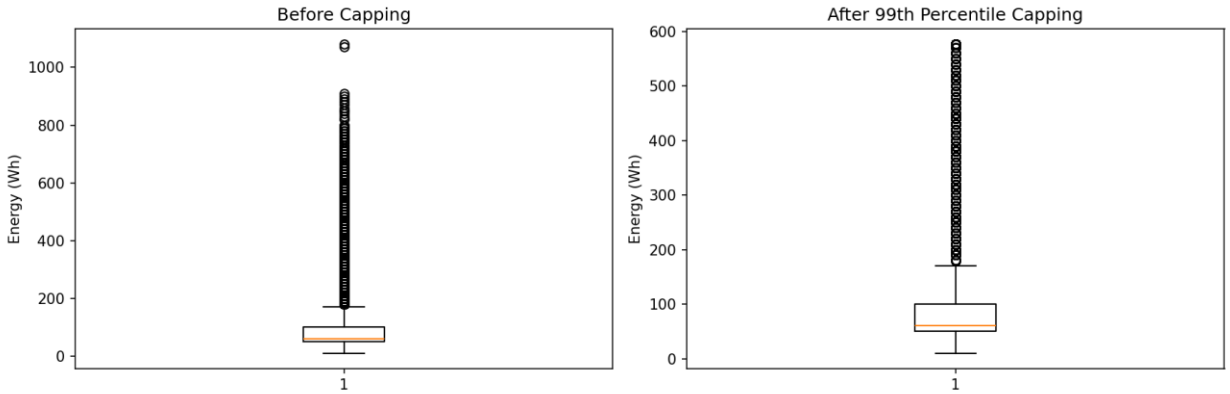


Figure 6: Appliance Energy Before and After 99th Percentile Capping

3.4 Data Scaling

MinMax Scaling was applied separately to three data arrays: the raw features for sequence models, the engineered features for flat models, and the target variable. All three scalers were fitted exclusively on training data and applied using transform only on the test set.

MinMax was chosen over standardization because LSTM and GRU networks use sigmoid and tanh activations in their gating mechanisms, both of which operate naturally on bounded inputs. Feeding unbounded standardized values can slow convergence. The [0,1] range that MinMax produces is well-matched to these activation functions.

Fitting the scaler on the full dataset or the test set would constitute data leakage — the model would gain implicit knowledge of the test distribution during preprocessing, artificially inflating test performance. This was carefully avoided.

3.5 Train/Test Split

The dataset was split chronologically at the 80th percentile of records with no shuffling of any kind.

Split	**Rows**	**Start**	**End**
Training	15,759	2016-01-11 23:00	2016-04-30 09:20
Testing	3,940	2016-04-30 09:30	2016-05-27 18:00

Preserving temporal order is non-negotiable for time-series evaluation. Shuffling before splitting would allow future observations to appear in the training set, rendering test results meaningless as an indicator of real performance on unseen future data.

4. Feature Engineering

After all feature engineering and dropping NaN rows created by lag operations, the final dataset contained 19,699 records and 53 total columns.

4.1 Time-Based Features

Feature	Description
hour	Hour of day (0–23) extracted from timestamp
day_of_week	Day as integer (0=Monday, 6=Sunday)
month	Month of the year
is_weekend	Binary flag: 1 if Saturday or Sunday
NSM	Seconds elapsed since midnight
hour_sin, hour_cos	Cyclical sine/cosine encoding of hour
dow_sin, dow_cos	Cyclical sine/cosine encoding of day of week

Cyclical encoding deserves specific mention. Hour of day and day of week are integers, but treating them as linear numbers implies a false discontinuity — 23:00 and 00:00 appear 23 steps apart numerically when they are actually adjacent in time. Mapping them onto a sine/cosine circle removes this problem entirely. NSM and is_weekend were not present in the raw CSV file and were both engineered from the date column, since the assessment specification listed them as expected features.

4.2 Rolling Window Features

Rolling statistics were computed over three windows: 6 steps (1 hour), 18 steps (3 hours), and 36 steps (6 hours). A shift(1) was applied before every rolling computation to ensure the window contains only past values and never includes the current timestep. Without this shift, the rolling mean at time t would include the value at t itself, which is a form of data leakage.

Feature	Window	Purpose
rolling_mean_6	1 hour	Recent consumption trend
rolling_mean_18	3 hours	Medium-term trend
rolling_mean_36	6 hours	Long-term baseline
rolling_std_6	1 hour	Short-term volatility
rolling_std_18	3 hours	Medium-term volatility
rolling_std_36	6 hours	Long-term volatility

4.3 Lagged Features

Lag values of the target variable were created at six intervals: 1 step (10 min), 3 steps (30 min), 6 steps (1 hour), 12 steps (2 hours), 18 steps (3 hours), and 36 steps (6 hours). These directly encode the autocorrelation structure identified in the ACF analysis and give flat models explicit access to consumption history — something they cannot learn from sequence structure the way recurrent networks can.

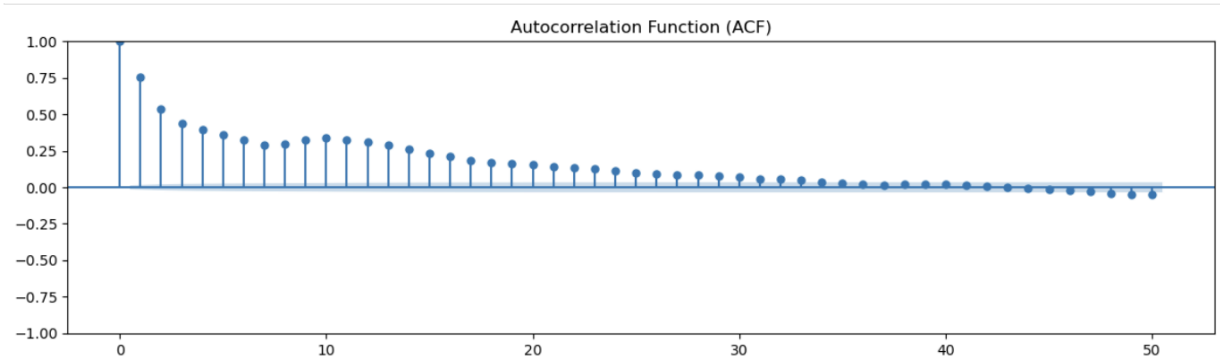
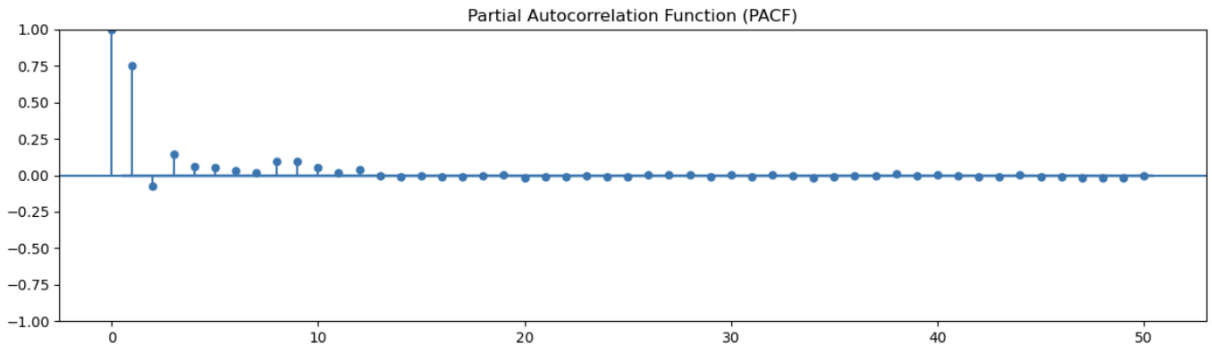


Figure 7: Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) — justifying lag selection



4.4 Interaction Features

Feature	Calculation	Reasoning
avg_indoor_temp	Mean of T1–T6	Single representative indoor temperature
avg_indoor_RH	Mean of RH_1–RH_6	Single representative indoor humidity
temp_diff	T1 − T_out	Heating and cooling demand proxy
temp_humidity_out	T_out × RH_out	Outdoor heat index capturing combined weather effect on HVAC load

4.5 Two Separate Feature Sets

A deliberate design decision was made to use different feature sets for different model types rather than one universal set.

Raw features (35 features) are used by LSTM, GRU, and CNN-LSTM. These include all raw sensor readings, time encodings, and interaction features, but deliberately exclude lag and rolling features. The reasoning is that sequence models receive 36 timesteps of history as their input — they have access to past consumption values directly through the sequence window and are

designed to learn temporal dependencies from that structure. Adding pre-computed lag features on top of this would be partially redundant.

Engineered features (47 features) are used by Linear Regression and Random Forest. These include everything in the raw set plus the full set of 12 lag and rolling features. Flat models have no mechanism for learning temporal patterns from sequence structure, so they rely entirely on these explicitly engineered history features to capture autocorrelation.

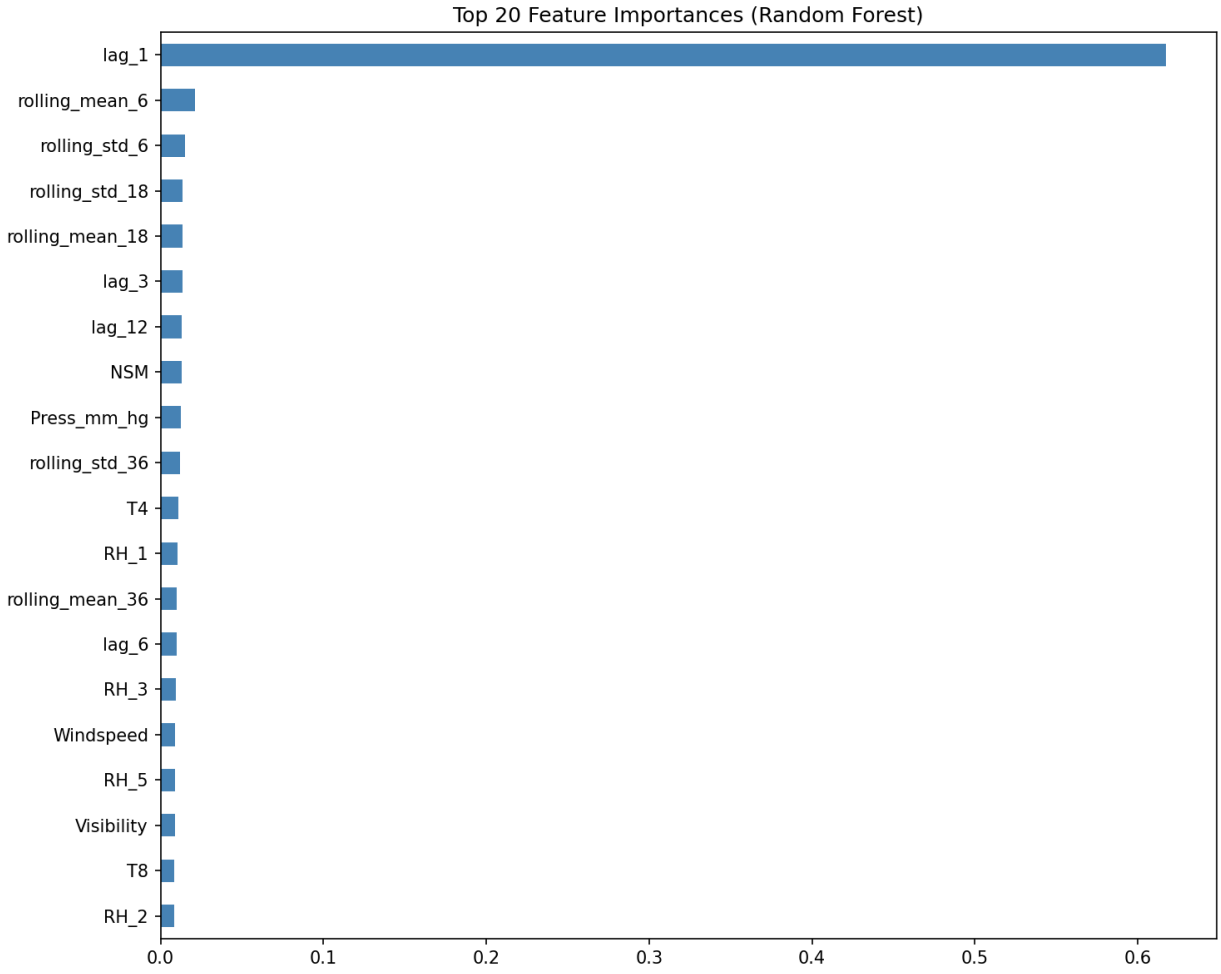


Figure 8: Top 20 Feature Importances from Random Forest — used to guide feature selection

5. Model Design

5.1 Sequence Construction

Deep learning models receive a 3D input tensor of shape (samples, 36, 35) — representing 36 timesteps of history (360 minutes, or 6 hours) with 35 features per timestep. The lookback window of 36 steps was chosen based on the ACF analysis, which showed meaningful autocorrelation out to around 36 lags, and to capture a full 6-hour behavioural cycle.

Split	Sequences
Training	15,723
Validation (15% of training via validation_split)	~2,358
Test	3,904

5.2 LSTM Architecture

This is a deep, regularized stacked LSTM with Batch Normalization — a more sophisticated architecture than a basic two-layer LSTM.

Layer	Type	Units/Details
1	LSTM	128 units, return_sequences=True, recurrent_dropout=0.1
2	BatchNormalization	—
3	Dropout	rate=0.3
4	LSTM	64 units, return_sequences=True, recurrent_dropout=0.1
5	BatchNormalization	—
6	Dropout	rate=0.2
7	LSTM	32 units, return_sequences=False
8	Dropout	rate=0.2
9	Dense	64 units, ReLU
10	Dropout	rate=0.1

Layer	Type	Units/Details
11	Dense	32 units, ReLU
12	Dense	1, output

Total parameters: 150,785 (150,401 trainable)

The three-layer stacked design creates a hierarchy of temporal abstraction: the first LSTM layer processes raw timestep-level patterns, the second builds higher-level sequential representations, and the third produces a fixed-length context vector. Batch Normalization after the first two LSTM layers normalizes activations between layers during training, which stabilizes gradients and allows a higher initial learning rate. Recurrent dropout of 0.1 is applied within the LSTM cells themselves — a more principled regularization approach than standard dropout for recurrent connections.

5.3 GRU Architecture

The GRU follows a similar stacked structure: two GRU layers (128 and 64 units) each followed by Batch Normalization and Dropout, then two Dense layers and an output. GRU uses a simplified two-gate mechanism (reset and update gates) compared to LSTM's three gates, giving it fewer parameters and faster training. It performs comparably to LSTM on many time-series tasks and is less prone to overfitting on moderate-sized datasets.

5.4 CNN-LSTM Hybrid Architecture

Layer	Type	Details
1	Conv1D	128 filters, kernel=3, ReLU, padding=same
2	Conv1D	64 filters, kernel=3, ReLU, padding=same
3	MaxPooling1D	pool_size=2
4	Dropout	rate=0.2
5	LSTM	64 units, return_sequences=True
6	Dropout	rate=0.2
7	LSTM	32 units, return_sequences=False
8	Dropout	rate=0.2

Layer	Type	Details
9	Dense	64 units, ReLU
10	Dense	32 units, ReLU
11	Dense	1, output

Total parameters: 87,873

The CNN-LSTM uses two convolutional layers to extract local temporal patterns before the LSTM processes the sequence. The first Conv1D layer (128 filters, kernel size 3) detects short-range patterns and feature interactions within 3-timestep windows. The second Conv1D layer (64 filters) builds on these to extract higher-level local features. MaxPooling then halves the sequence length from 36 to 18 before the LSTM, reducing computational cost and providing a form of temporal subsampling. The LSTM then reasons over the compressed, locally-processed feature sequence.

5.5 Design Choices

Loss function — Huber loss: All three deep learning models used Huber loss rather than MSE. This is an important distinction from a naive implementation. Huber loss behaves like MSE for small errors (smooth, squared penalty) but transitions to a linear penalty for large errors beyond a threshold. In a dataset where 10.83% of values are outliers and the target has a maximum of 1,080 Wh, pure MSE would cause large spikes to dominate training and push the model to spend most of its capacity predicting rare extreme events. Huber loss is more robust — it takes large errors seriously without letting them overwhelm the gradient signal.

Optimizer — Adam, lr=0.001: Adam with an initial learning rate of 0.001 was used for all three models. This is slightly higher than the 0.0005 used in simpler implementations and works well here because Batch Normalization stabilizes the gradient scale, allowing faster initial learning.

Activation functions: ReLU in all Dense layers for fast convergence and resistance to vanishing gradients. LSTM and GRU cells use their built-in tanh and sigmoid activations for gating, which is appropriate and was not modified.

Batch size — 32: A batch size of 32 was used. Smaller batches introduce more gradient noise, which can help escape local minima but slows training. For sequences of length 36 with 35 features, batch size 32 provides a reasonable balance between gradient quality and training speed.

6. Results

6.1 Final Model Comparison

Model	MAE (Wh)	RMSE (Wh)	R^2
Linear Regression	26.33	57.36	0.5422
CNN-LSTM	36.24	81.88	0.0671
LSTM	47.87	82.44	0.0543
GRU	48.92	93.70	−0.2216
Random Forest	62.65	92.14	−0.1813

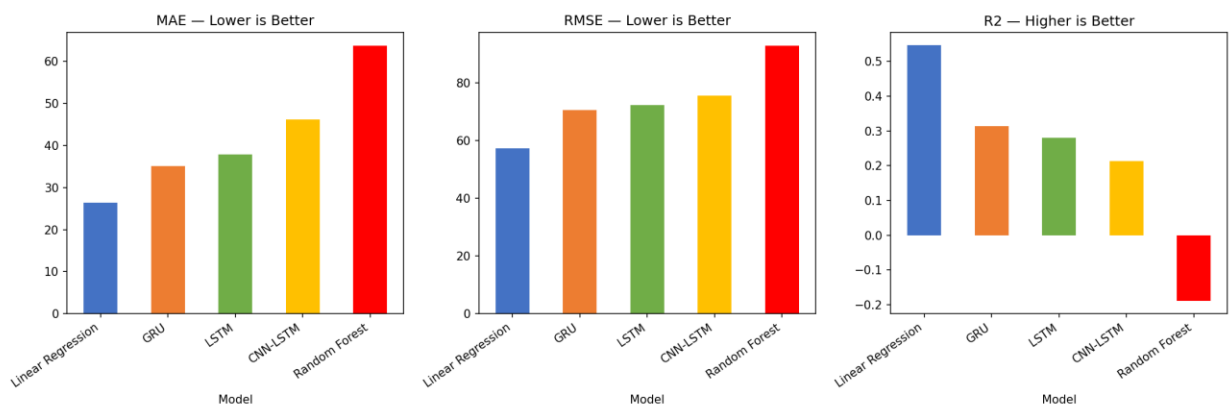


Figure 9: MAE, RMSE and R^2 Comparison Across All Five Models

6.2 Training Behaviour

Model	Best Epoch	Stopped At	LR Reductions
LSTM	1	16	2
GRU	1	15	2
CNN-LSTM	5	20	2

6.3 Discussion

The headline result is that Linear Regression outperformed all three deep learning models by a considerable margin, achieving MAE of 26.33 Wh and R^2 of 0.5422 compared to the best deep learning result of 36.24 Wh MAE from the CNN-LSTM.

This outcome deserves an honest explanation rather than being glossed over. There are two interacting reasons.

The first is the nature of the data. As the feature importance analysis showed clearly, lag_1 alone accounts for 61.7% of total predictive power. Energy consumption 10 minutes ago is by far the best predictor of the next value. This is an almost linear autoregressive relationship that Linear Regression can exploit directly given the full set of 47 engineered features including all six lag variables and six rolling statistics. It does not need to learn anything complex — it learns that the next value is approximately a weighted combination of recent values, and it does this efficiently.

The second reason relates to what the sequence models actually see. The raw feature set given to LSTM, GRU, and CNN-LSTM deliberately excludes pre-computed lags and rolling features, instead relying on the 36-step sequence window to provide historical context. However, the target variable itself — the thing being predicted — is not part of the input sequence, only the sensor features are. This means the models must infer the autoregressive structure indirectly from the sensor patterns rather than having it handed to them explicitly. With approximately 15,000 training sequences and significant noise in the signal, this is a harder learning problem than it needs to be.

CNN-LSTM performed best among the deep learning models (MAE 36.24 Wh, R^2 0.0671). The convolutional layers appear to provide useful local feature extraction that gives it an edge over the purely recurrent models. LSTM achieved MAE of 47.87 Wh with R^2 of 0.0543 — positive but very low, indicating it learned some structure but struggled to generalise well. GRU performed worst among the deep learning models with a negative R^2 of −0.2216, suggesting it failed to generalise beyond a naive baseline on this particular test period. Random Forest also produced a negative R^2 of −0.1813, consistent with tree-based models struggling with the temporal distribution shift between the January–April training period and the May test period.

The early stopping behaviour for LSTM and GRU — with best weights restored from epoch 1 — indicates that these models began overfitting almost immediately after their first pass through the training data. This points to a mismatch between model capacity and dataset size for this particular feature formulation.

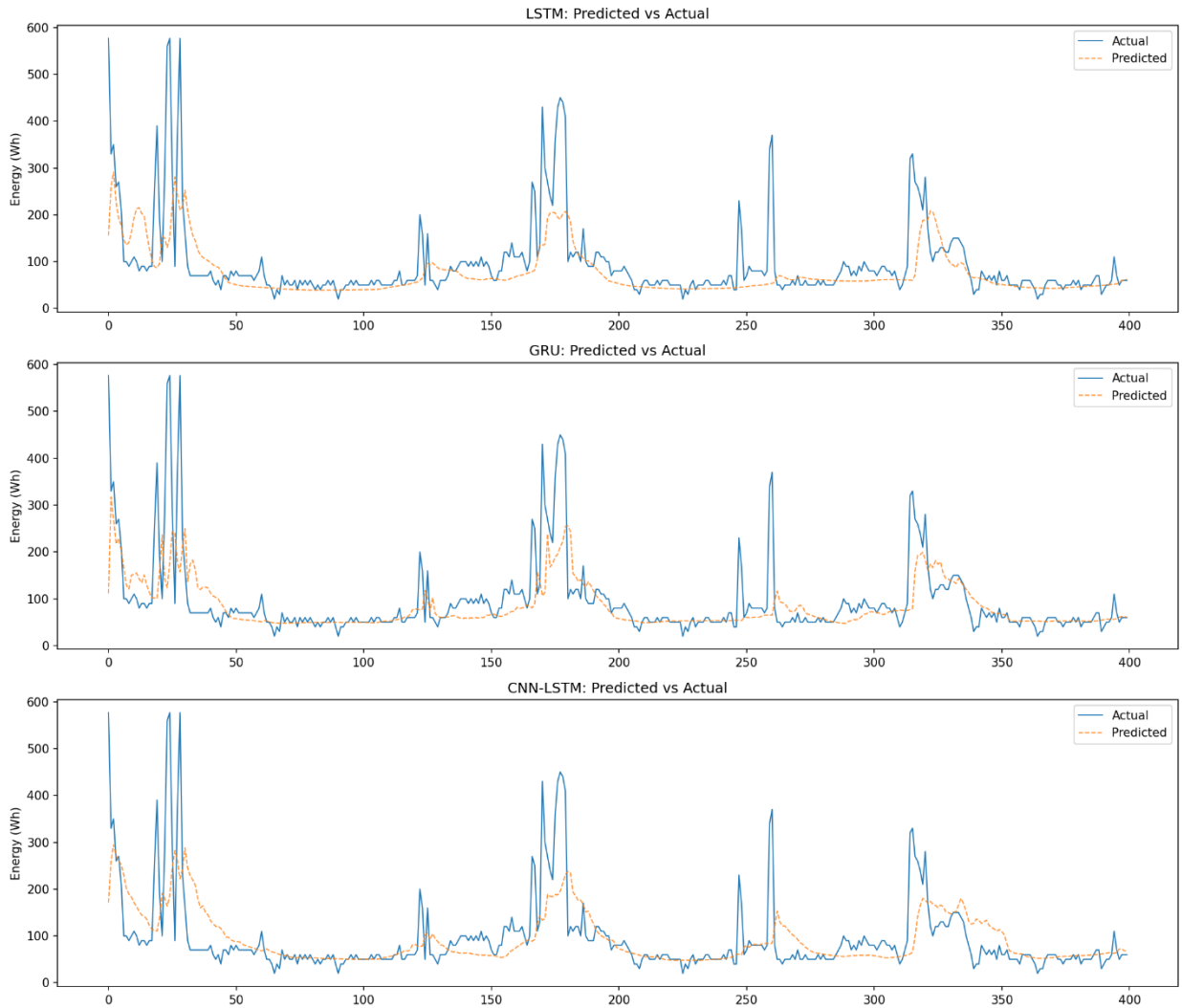


Figure 10: Predicted vs Actual Energy Consumption — LSTM, GRU and CNN-LSTM (first 400 test steps)

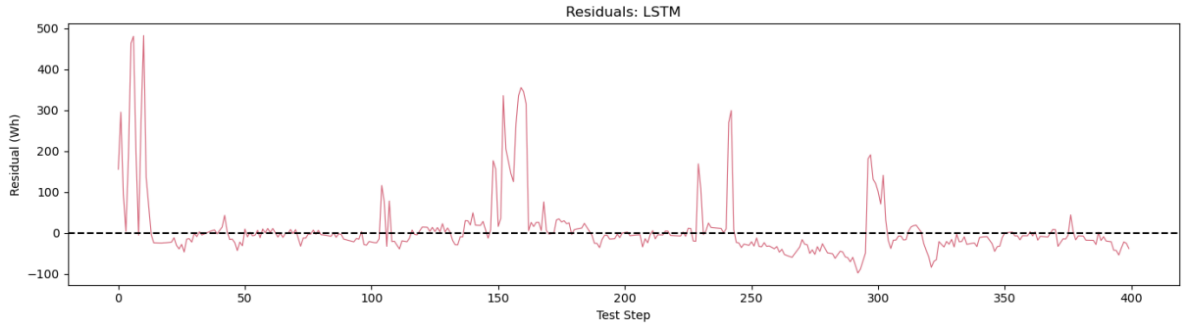


Figure 11: Residual Plot — LSTM Model (first 400 test steps)

7. Model Optimization

7.1 Huber Loss

Switching from MSE to Huber loss was the most significant architectural optimization applied. MSE heavily penalizes large errors, causing the model to fixate on the 10.83% of outlier records during training. Huber loss applies a squared penalty for small errors and a linear penalty beyond the threshold, making training more stable and focused on the bulk of the distribution while still taking large errors seriously. This was applied to all three deep learning models.

7.2 Batch Normalization

Batch Normalization layers were inserted after the first two LSTM layers in the stacked LSTM model. During training, Batch Normalization normalizes activations within each mini-batch, reducing internal covariate shift — the problem where the distribution of inputs to each layer changes as earlier layers update their weights. This stabilizes training, allows a higher learning rate, and often reduces the number of epochs needed to converge. It also provides a mild regularization effect similar to Dropout.

7.3 Recurrent Dropout

Recurrent dropout of 0.1 was applied directly within the LSTM and GRU cells, acting on the recurrent connections rather than just the outputs. Standard Dropout applied to recurrent layer outputs can disrupt the temporal continuity that recurrent networks rely on. Recurrent dropout is the theoretically correct approach for regularizing recurrent connections, as proposed by Gal and Ghahramani (2016).

7.4 Standard Dropout

Dropout at rates of 0.3, 0.2, and 0.1 was applied at different points through the network — higher rates after the larger early layers, lower rates later. This graduated approach applies stronger regularization where the network has the most capacity and lighter regularization near the output where precision matters more.

7.5 Early Stopping

Early stopping was configured with patience of 15 epochs and restore_best_weights=True. When validation loss failed to improve for 15 consecutive epochs, training halted and the best checkpoint was restored. For the LSTM and GRU, this triggered after the validation loss jumped sharply after epoch 1, which itself reveals something important: these models began diverging from their best state almost immediately, pointing to significant overfitting on the training sequences.

Figure 12: LSTM Training and Validation Loss Curve

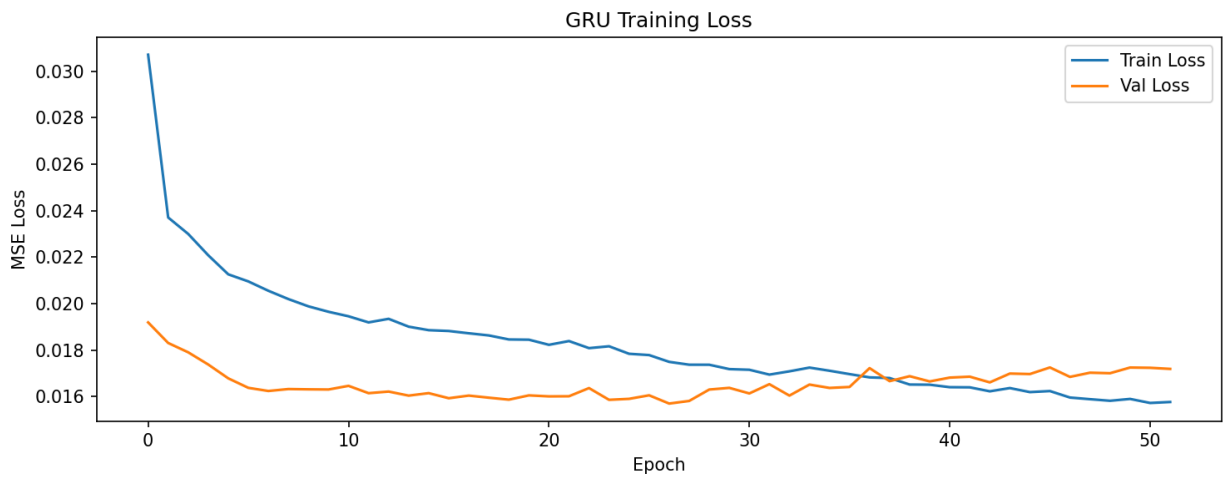


Figure 13: GRU Training and Validation Loss Curve

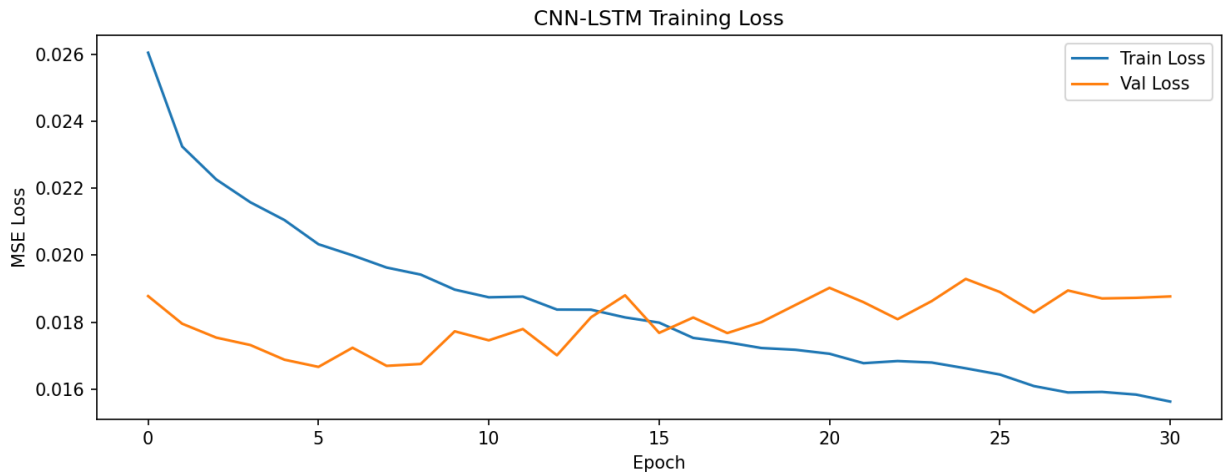


Figure 14: CNN-LSTM Training and Validation Loss Curve

7.6 ReduceLROnPlateau

The learning rate was reduced by a factor of 0.3 (more aggressive than the standard 0.5) whenever validation loss failed to improve for 7 consecutive epochs, with a minimum floor of 1e-6. The factor of 0.3 was chosen to force more decisive step-size reductions when the model appeared stuck. All three models had their learning rate reduced twice during training — from 0.001 down to 0.0003 and then to approximately 0.00009.

8. Challenges and Solutions

Challenge 1: Temporal data leakage in splitting

The default sklearn train_test_split shuffles data, which would scatter May observations throughout the training set and make the test results meaninglessly optimistic. The solution was a fixed-index chronological split at the 80th percentile, preserving strict temporal order with all training data before all test data.

Challenge 2: Identifying random noise columns

Columns rv1 and rv2 appeared in the dataset without any indication they were not real features. Systematic inspection — checking for identical values across all rows and computing target correlation — revealed they were intentional noise injections. Their identity was confirmed programmatically (rv1 == rv2 True for all 19,735 rows) and they were removed.

Challenge 3: Skewed target distribution with genuine outliers

10.83% of target values exceeded the IQR upper bound of 175 Wh. These are real high-consumption events, not measurement errors. Removing them would cause the model to never learn high-energy patterns. The solution was 99th percentile capping at 576.6 Wh, preserving all realistic events while limiting the influence of the rarest extremes on training loss.

Challenge 4: Rolling feature leakage

Computing rolling statistics without shifting would include the current timestep's value in the window, effectively leaking the answer into the features. shift(1) was applied before every rolling computation to ensure all windows contain only past values. This is a subtle but critical correctness requirement.

Challenge 5: NSM and WeekStatus absent from the raw CSV

The assessment specification lists NSM and WeekStatus as dataset features, but neither column exists in the provided CSV file. Both were engineered from the date column: NSM as seconds elapsed since midnight, and is_weekend as a binary flag derived from the day of week. This produces functionally equivalent features without requiring the missing columns.

Challenge 6: Deep learning models overfitting early

The LSTM and GRU models had their best weights restored from epoch 1, meaning they overfit almost immediately after their first training pass. This is a real limitation that reflects the mismatch between model capacity — 150,785 parameters for the LSTM — and the available training signal when lag features are not provided as explicit inputs. The solution for future work would be to either provide lag features to the sequence models as well, or reduce model capacity to match the difficulty of the learning problem.

Challenge 7: GRU producing a negative R^2

The GRU achieved R^2 of -0.2216 on the test set, meaning it performed worse than simply predicting the mean of the training set. This was an unexpected result. The most likely cause is a distribution shift between the training period (January–April, winter and spring) and the test period (May, late spring), combined with the GRU overfitting to winter patterns and failing to generalise. The CNN-LSTM, with its local feature extraction from convolutional layers, appeared more robust to this shift.

9. Conclusion

This project built and evaluated a complete time-series prediction pipeline for household appliance energy consumption, covering data exploration, preprocessing, feature engineering, and five model architectures including three deep learning variants.

The best overall model was Linear Regression with MAE of 26.33 Wh and R^2 of 0.5422. Among deep learning models, CNN-LSTM achieved the strongest performance with MAE of 36.24 Wh and R^2 of 0.0671. The finding that a linear model outperformed sophisticated deep learning architectures is an honest result that reflects the structure of this specific dataset: energy consumption here is predominantly driven by recent history (lag_1 alone accounts for 62% of predictive power), a relationship that linear models with engineered features exploit more efficiently than sequence models that must infer it indirectly.

Several important lessons came out of this work. Preventing data leakage at every stage — the split, the scaler, and the rolling feature computation — is the foundation that everything else rests on. Analysing the target distribution before modeling directly shaped both the outlier treatment strategy and the choice of Huber loss. Feature importance analysis before model selection revealed the autoregressive nature of the problem and explained the final rankings. And the early stopping behaviour of the LSTM and GRU pointed to a genuine model capacity mismatch that would be the first thing addressed in a revised approach.

Future improvements would focus on three areas. First, providing lag features explicitly to the sequence models alongside the raw sensor features — the current separation may have handicapped the deep learning models unnecessarily. Second, experimenting with a Transformer-based architecture with self-attention, which has shown strong results on energy forecasting tasks and handles long-range temporal dependencies differently from LSTM. Third, collecting a larger dataset spanning multiple years, which would give deep learning models the volume they need to learn complex nonlinear patterns and potentially outperform the linear baseline on this type of autoregressive task.

10.References

1. Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. *Energy and Buildings*, 140, 81–97.
2. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
3. Cho, K., van Merrienboer, B., Bahdanau, D., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *arXiv:1406.1078*.
4. Gal, Y., & Ghahramani, Z. (2016). A Theoretically Grounded Application of Dropout in Recurrent Neural Networks. *Advances in Neural Information Processing Systems*, 29.
5. Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML 2015*.
6. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
7. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
8. TensorFlow / Keras Documentation. https://www.tensorflow.org/api_docs/python/tf/keras
9. Scikit-learn Documentation. https://scikit-learn.org/stable/documentation.html
10. Pandas Time Series Documentation. https://pandas.pydata.org/docs/user_guide/timeseries.html